Informe de auditoría de seguridad

Aplicación: Historias Clínicas HDW (Java 21 · Spring Boot 4 · Thymeleaf · PostgreSQL)
Fecha: 4 de noviembre de 2026 · Archivo revisado: hwl-registro-con-codigo_v2.zip

1. Resumen

El código está bien hecho: no tiene inyección SQL, ni XSS, y cada usuario sólo ve sus propios datos. El problema más grave no está en el código sino en el ZIP: trae contraseñas reales. Después de eso, lo más urgente es ponerle freno a los intentos de inicio de sesión.

CRÍTICO	ALTO	MEDIO	BAJO
1	2	6	4

2. Qué se probó y resultado

Prueba	Resultado	Por qué
Inyección SQL	SIN FALLAS	Todas las consultas usan Spring Data JPA (métodos derivados y @Query con :parámetros). No hay SQL armado concatenando texto. Un ' OR 1=1 -- se trata como texto normal.
XSS (inyectar código en la página)	SIN FALLAS	Thymeleaf usa th:text (escapa todo), no hay th:utext. El JavaScript usa textContent. El correo HTML escapa el nombre.
CSRF (acciones falsas desde otra web)	SIN FALLAS	Spring Security lo trae activo; todos los formularios usan th:action (token incluido) y el chat reutiliza ese token.
IDOR (ver datos de otro paciente cambiando el id)	SIN FALLAS	Borrar/cancelar busca por id Y dueño (findByIdAndUsuarioId). El PDF sólo genera el historial propio.
Control de acceso / rol admin	SIN FALLAS	/admin/** exige ROLE_ADMIN; el resto exige sesión + doble factor.
Guardado de contraseñas	SIN FALLAS	BCrypt. Códigos y tokens con SecureRandom, guardados como hash SHA-256 y comparados en tiempo constante.
Doble factor (MFA)	MEJORABLE	Bien implementado y cifrado, pero permite reusar el código (H-07) y su clave tiene valor por defecto (H-03).
Fuerza bruta / límite de intentos	FALLA	El login no tiene límite (H-02); el código de registro se reinicia (H-05).
Secretos y configuración	FALLA	Contraseñas reales en el ZIP (H-01); configuración de desarrollo (H-08).
Enumeración de usuarios	FALLA LEVE	El registro revela si un correo existe (H-06).
Transporte (HTTPS / cookies)	FALLA	No se fuerza HTTPS ni cookies Secure (H-04).

Subida de archivos / rutas / comandos del sistema	**NO APLICA**	La app no sube archivos ni ejecuta comandos ni lee rutas que vengan del usuario.

Método: revisión manual de todo el código fuente (controladores, servicios, consultas, seguridad, plantillas y JavaScript). Es una revisión del código

3. Hallazgos en detalle

H-01 · Contraseñas reales dentro del ZIP	**CRÍTICO**
Dónde	hwl-main/.env y .vscode/launch.json
Qué pasa	El ZIP compartido incluye el archivo .env con valores reales: contraseña de la base de datos, contraseña de aplicación de Gmail, contraseña del administrador inicial y las claves de cifrado del doble factor. La contraseña de Gmail aparece además escrita a mano en launch.json. El .gitignore los excluye de Git, pero no de un ZIP.
Qué podría hacer un atacante	Cualquiera que reciba el ZIP puede entrar a la base de datos con historias clínicas, enviar correos falsos desde la cuenta Gmail del proyecto, entrar como administrador y descifrar los secretos del Authenticator.
Cómo arreglarlo	1) Cambiar HOY todas esas contraseñas (base de datos, contraseña de aplicación de Google, admin) y generar claves MFA nuevas. 2) Borrar el bloque env de launch.json. 3) Compartir sólo un .env.example con valores vacíos. 4) Antes de comprimir, excluir .env, .vscode y target/.

H-02 · Sin límite de intentos en el inicio de sesión	**ALTO**
Dónde	SecurityConfig.java (formLogin)
Qué pasa	El doble factor tiene bloqueo tras 5 fallos, pero la pantalla de usuario y contraseña no tiene ninguno. Además, la respuesta cambia según la contraseña sea correcta (lleva a /mfa) o no (/login?error).
Qué podría hacer un atacante	Probar millones de contraseñas (fuerza bruta o listas filtradas) sin freno y saber exactamente cuál es la correcta. Con eso sólo le falta el código del celular.
Cómo arreglarlo	Contar fallos por correo y por IP (p. ej. 5 fallos = 15 min de bloqueo, guardado en la base de datos), agregar retraso creciente o CAPTCHA, y avisar al usuario por correo de intentos sospechosos.

H-03 · Clave de cifrado del doble factor con valor por defecto en el código	**ALTO**
Dónde	application.properties: app.mfa.encryption-salt
Qué pasa	Si la variable MFA_ENCRYPTION_SALT no se define, se usa un salt fijo escrito en el código (5c07...). El archivo indica que la contraseña de cifrado debe venir del entorno, pero el valor del .env es una frase legible y débil.
Qué podría hacer un atacante	Si alguien obtiene la base de datos y el código (que es público para quien tenga el ZIP), puede descifrar los secretos TOTP y generar códigos del Authenticator de todos los usuarios.
Cómo arreglarlo	Quitar los valores por defecto para que la app NO arranque sin ellos. Usar valores largos y aleatorios (32+ bytes, p. ej. openssl rand -hex 32) guardados en un gestor de secretos.

H-04 · Sin HTTPS forzado ni cookies seguras	MEDIO
Dónde	application.properties / SecurityConfig
Qué pasa	No se exige HTTPS, la URL base por defecto es http:// y no se configuran las cookies de sesión como Secure ni SameSite=Strict.
Qué podría hacer un atacante	En una red Wi-Fi pública se puede robar la cookie de sesión o la contraseña al iniciar sesión y entrar a la historia clínica de la víctima.
Cómo arreglarlo	Desplegar siempre detrás de HTTPS, agregar server.servlet.session.cookie.secure=true, same-site=strict, activar HSTS y redirigir http a https.

H-05 · El código de registro se puede adivinar poco a poco	MEDIO
Dónde	CodigoRegistroService.reenviar()
Qué pasa	Cada vez que se pide un código nuevo, el contador de intentos vuelve a 0. Así se pueden hacer 5 intentos por minuto sin fin (y abrir sesiones nuevas da más intentos).
Qué podría hacer un atacante	Con paciencia o varias sesiones, crear una cuenta usando el correo de otra persona sin tener acceso a ese correo.
Cómo arreglarlo	Limitar intentos por correo y por IP en la base de datos (no en la sesión), poner un máximo de reenvíos por hora y no reiniciar el contador al reenviar.

H-06 · Se puede saber qué correos están registrados	MEDIO
Dónde	CodigoRegistroService.iniciar() → mensaje “Ya existe una cuenta con ese correo”
Qué pasa	El formulario de registro dice claramente si un correo ya tiene cuenta. (La pantalla de reenviar sí lo oculta bien.)
Qué podría hacer un atacante	Armar una lista de pacientes de la clínica (dato sensible en sí mismo) y usarla para phishing o para atacar sus contraseñas.
Cómo arreglarlo	Responder siempre igual (“Te enviamos un código”) y, si el correo ya existe, mandarle a ese correo un aviso en vez del código.

H-07 · El código del Authenticator se puede reutilizar	MEDIO
Dónde	MfaService.codigoValido()
Qué pasa	Un mismo código de 6 dígitos sigue siendo válido durante su ventana de tiempo (unos 30-90 s), aunque ya se haya usado.
Qué podría hacer un atacante	Si alguien ve o intercepta el código (por encima del hombro, phishing en tiempo real), puede usarlo de nuevo justo después.
Cómo arreglarlo	Guardar el último intervalo de tiempo usado por cada usuario y rechazar códigos iguales o anteriores.

H-08 · Configuración de desarrollo que podría llegar a producción	MEDIO

Dónde	application.properties
Qué pasa	ddl-auto=update (Hibernate cambia la base solo), thymeleaf.cache=false, y si falta DB_URL la app arranca con una base H2 en memoria sin avisar.
Qué podría hacer un atacante	Cambios no controlados en la estructura de la base, peor rendimiento, o una app en producción guardando historias en memoria que se pierden al reiniciar.
Cómo arreglarlo	Crear un perfil prod con ddl-auto=validate, caché activada y sin valores por defecto para la base de datos (que falle si falta). Usar migraciones (Flyway).

H-09 · Bloqueos guardados sólo en memoria — MEDIO

Dónde	MfaIntentos.java, AsistenteCitasService (mensajesRecientes)
Qué pasa	Los contadores de intentos viven en la RAM: se borran al reiniciar, no se comparten entre servidores y crecen sin límite.
Qué podría hacer un atacante	Saltarse el bloqueo del doble factor provocando o esperando un reinicio; con muchos correos distintos, llenar la memoria del servidor.
Cómo arreglarlo	Guardar los intentos en la base de datos o Redis, con caducidad automática (p. ej. Caffeine/Bucket4j con tamaño máximo).

H-10 · Política de contraseñas mínima — BAJO

Dónde	UsuarioService / CodigoRegistroService
Qué pasa	Sólo se exige 8 caracteres; no hay máximo (BCrypt ignora lo que pase de 72 bytes) ni se bloquean contraseñas comunes como 12345678.
Qué podría hacer un atacante	Usuarios con contraseñas fáciles que caen rápido ante el ataque del punto H-02.
Cómo arreglarlo	Mínimo 10-12 caracteres, máximo 72, y rechazar contraseñas de listas filtradas (p. ej. API de HaveIBeenPwned por k-anonimato).

H-11 · Datos personales en los registros (logs) — BAJO

Dónde	CodigoRegistroService, VerificacionCorreoService
Qué pasa	Se escriben correos de pacientes en los logs; en modo consola también el código o enlace de verificación.
Qué podría hacer un atacante	Quien lea los logs obtiene datos personales o puede verificar cuentas ajenas. Para datos de salud aplica la Ley 1581 de 2012 (Colombia).
Cómo arreglarlo	Enmascarar correos en logs (j***@gmail.com) y asegurar que MAIL_MODO_CONSOLA nunca esté en true fuera de tu computador.

H-12 · Asistente con IA: manipulación por texto (prompt injection) — BAJO

Dónde	AsistenteCitasService
Qué pasa	Un usuario puede escribir instrucciones para que la IA diga cosas fuera de tema. Está bien mitigado: la IA sólo propone, el servidor valida la cita y sólo afecta al propio usuario.

Qué podría hacer un atacante	Respuestas inapropiadas o gasto de la cuota de la API de Gemini.
Cómo arreglarlo	Mantener la validación actual y añadir un límite diario de mensajes por usuario.

H-13 · Faltan cabeceras de seguridad extra y código sin usar — BAJO

Dónde	SecurityConfig / UsuarioService.registrar, flujo de enlace por correo
Qué pasa	No hay Content-Security-Policy propia. Quedan dos sistemas de verificación de correo (código y enlace) y métodos que ya no se usan.
Qué podría hacer un atacante	Más superficie de ataque y más fácil cometer errores al mantener el código.
Cómo arreglarlo	Agregar CSP (default-src 'self'), borrar el flujo que no se use y revisar dependencias con OWASP Dependency-Check.

4. Plan de acción recomendado

Cuándo	Qué hacer	Hallazgos
Hoy	Cambiar todas las contraseñas del .env, generar claves MFA nuevas, limpiar launch.json y dejar de compartir el .env.	H-01
Esta semana	Límite de intentos en el login, quitar valores por defecto de las claves, HTTPS + cookies seguras.	H-02, H-03, H-04
Este mes	Corregir el reenvío del código, mensajes iguales en el registro, evitar reuso del código MFA, perfil de producción, bloqueos en base de datos.	H-05 a H-09
Cuando se pueda	Política de contraseñas, logs sin datos personales, CSP, limpiar código sin uso, revisar dependencias.	H-10 a H-13